

ZSpeed Fleet Operations

Connected Vehicle Intelligence Solution

Submission format: PDF export of this document

Team name: _____

Team members and roles: _____

Contact email: _____

Problem space: Fleet telemetry quality and vehicle operations

GitHub repository URL: _____

Demo video URL: _____

Submission date: _____

Table of Contents

- [1. Executive Summary](#)
- [2. Problem Statement & Validation](#)
 - [2.1 Problem Statement](#)
 - [2.2 Evidence & Validation](#)
 - [2.3 Impact & Success Metrics](#)
- [3. Solution Description](#)
 - [3.1 Solution Overview & User Journey](#)
 - [3.2 Key Value Proposition](#)
 - [3.3 Innovative Ideas](#)
- [4. Feature List](#)
- [5. Solution Architecture \(High-Level Design\)](#)
 - [5.1 Architecture Overview](#)
 - [5.2 Technology Stack & Justification](#)
 - [5.3 Data Architecture](#)
 - [5.4 Deployment View](#)
- [6. Low-Level Design](#)
 - [6.1 Layering & Separation of Concerns](#)
 - [6.2 Design Principles Applied](#)
 - [6.3 Design Patterns Used](#)
 - [6.4 Interfaces, Contracts & Runtime Flows](#)
 - [6.5 Algorithms & Data Structures](#)
- [7. Non-Functional Requirements & Performance Benchmarks](#)
- [8. Security & Compliance](#)
- [9. Test Strategy](#)
- [10. Observability](#)
- [11. AI / ML Component \(if used\)](#)
- [12. Architecture Decisions, Risks & Future Enhancements](#)
- [13. Demo Video \(5 Minutes Maximum\)](#)
- [14. Repository Checklist](#)
- [15. Conclusion](#)
- [16. Declarations](#)
- [17. Appendix \(if any\)](#)

1. Executive Summary

ZSpeed Fleet Operations gives fleet managers one place to onboard vehicles, connect OEM accounts, inspect vehicle health and trips, and restore readings held after an OEM changes its data format. We built a working fleet interface, platform backend, independent OEM simulator and simulator control console.

The repair workspace lets a manager match renamed fields, choose measurement units, translate status codes and decide how to handle added or removed readings. A preview checks the proposed changes before a versioned repair is enabled and valid historical records are restored. Original payloads and accepted history remain available.

Vehicle summaries and trip bundles are built in background jobs and served from a separate read database copy, with optional Redis caching. This removes repeated reconstruction from raw telemetry during page visits. The current implementation uses a durable SQLite queue and a replica on the same host. Kafka and a distributed database are proposed next steps.

2. Problem Statement & Validation

2.1 Problem Statement

A fleet manager needs usable vehicle and trip information when OEM payloads change because a renamed field, different unit or missing reading can otherwise interrupt reporting and require technical investigation. Operations staff need to keep valid data visible while the affected readings are reviewed.

Primary user: fleet manager. Other stakeholders: operations analysts, platform administrators and OEM integration teams. The prototype demonstrates data quality and recovery; fleet-wide financial or safety benefits have not been measured.

2.2 Evidence & Validation

Evidence or assumption	Method	Finding	Confidence
Format changes can block readings	Synthetic OEM changes and repair tests	Renames, units and status translations can be repaired locally	High for tested cases
Bad coordinates can break a map	Coordinate validation and map tests	Invalid points are handled without crashing vehicle views	High for tested cases
Saved trip bundles reduce read work	Read scaling tests and source review	Vehicle visits read saved summaries instead of rebuilding raw history	High structurally
Managers can complete repairs unaided	Guided browser walkthrough	Flow works end to end; independent user study still needed	Provisional

2.3 Impact & Success Metrics

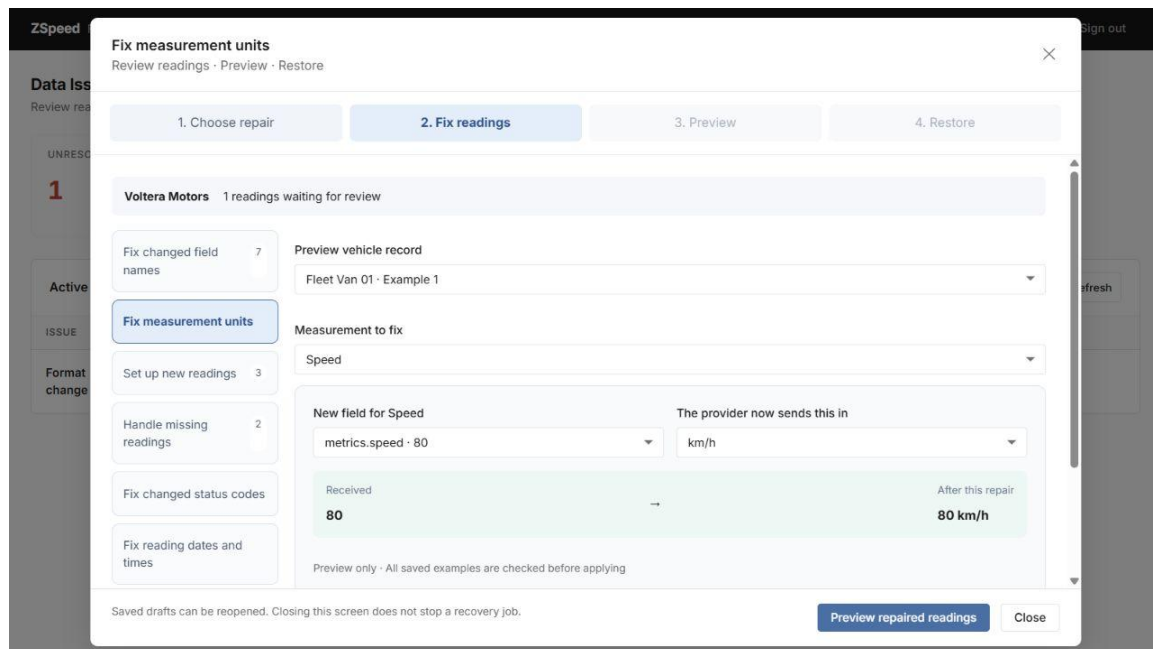
Success is measured by usable trip data, recovery outcomes, query cost and visible freshness. No 10,000 or 100,000 vehicle deployment, cost saving, throughput or percentile latency is claimed. A pilot should measure those outcomes against the existing raw-event lookup path.

3. Solution Description

3.1 Solution Overview & User Journey

A manager imports vehicles, confirms onboarding, connects an OEM account and chooses the readings to receive. The simulator then sends events. Accepted events update vehicle state and trips; held events appear as repair categories with concrete actions.

- Open a vehicle to see saved health details, a trip list and a route with important events and data quality information.
- Open the affected repair category. Match the old field to its new location, select the correct unit or translate the new status value.
- Preview the sampled readings. Resolve any failed checks, then choose Restore valid readings. Track the recovery by its job ID.



Working product example: the unit repair view shows the received speed, the selected unit and the converted value before the manager previews the repair.

3.2 Key Value Proposition

Managers can fix supported format changes inside the platform and recover usable history without writing transformation code. The interface keeps valid vehicle information available and reports what remains held. Conflicting identifiers and malformed records are separated from repairable mapping changes because they cannot safely be solved by choosing another field.

3.3 Innovative Ideas

Three implemented ideas distinguish this demonstrator: repair choices tailored to the failure category with before-and-after samples; versioned recovery that preserves raw payloads and accepted history;

and grouped background activity that turns many events for one vehicle into a readable job row. These are implementation choices, not claims of industry-first functionality.

4. Feature List

Done means implemented in the current code. Video timestamps below are suggested recording segments, to be confirmed after the demo is recorded. Code paths are relative to the repository; backend services are under `backend/src/services/`.

ID	Feature	User story	Priority	Status	Code path	Video
F01	CSV onboarding	Add vehicles with checked rows	Must	Done	<code>import.service.ts</code>	1:00
F02	OEM connections	Know which vehicles are receiving data	Must	Done	<code>connection.service.ts</code>	1:20
F03	Trip map and quality	Inspect usable routes and missing data	Must	Done	<code>read-model.service.ts</code>	1:45
F04	Guided format repair	Fix renamed fields and changed units	Must	Done	<code>MappingRepair.tsx</code> ; <code>mapping-repair.service.ts</code>	2:00
F05	Historical recovery	Restore valid held readings safely	Must	Done	<code>quarantine.service.ts</code>	3:00
F06	Grouped jobs	See progress for one vehicle activity	Should	Done	<code>background-activity.service.ts</code>	3:30
F07	Fast saved reads	Load saved trip details in one request	Must	Done	<code>read-model.service.ts</code> ; <code>read-cache.service.ts</code>	3:45
F08	Docker and AWS package	Start the packaged stack	Should	Partial	<code>docker-compose.yml</code> ; <code>deploy-aws.ps1</code>	4:00

F08 is packaged with launch scripts and health checks. A live Docker deployment and AWS deployment still require verification in an environment with the required tooling and account access.

5. Solution Architecture (High-Level Design)

5.1 Architecture Overview

System context: the fleet manager uses the fleet web application. An operator uses the simulator console to configure fictional OEM sources. The backend connects those sources to fleet-scoped ingestion, repair and read APIs. No live commercial OEM account or external identity provider is integrated in the demonstrator.

Container flow: Fleet UI → HTTP and Socket.IO → backend API. Simulator UI → HTTP → simulator backend → connector ingestion → operational SQLite. A supervised worker consumes durable jobs and updates a materialized read database → replica SQLite → optional Redis → fleet read API.

An event is retained, checked for duplication, normalized using its selected mapping and either accepted or quarantined. Accepted changes request coalesced projection work. The UI reads the latest completed snapshot; hop latency has not been benchmarked.

5.2 Technology Stack & Justification

Layer	Choice	Reason
Web interfaces	React, TypeScript, Vite	Shared typed models and separate fleet and simulator screens
Platform API	Node.js, Express, Socket.IO	HTTP workflows and fleet-scoped live updates
Processing	Supervised worker and durable SQLite jobs	Persisted retries and restart recovery without a broker dependency
Data and cache	SQLite WAL and optional Redis	Simple deployment; separate saved read views and bounded cache fallback
Mapping	Deterministic rules and versioned configurations	Predictable previews and repeatable validation
Deployment	Docker Compose, Caddy, AWS CloudFormation helper	Same stack locally or on a single cloud host

5.3 Data Architecture

The relational core associates fleets with vehicles and OEM connections. Connections own raw events and mapping versions. Accepted measurements feed vehicle state, trips and routes; held records belong to quarantine incidents and recovery jobs. Raw payloads remain the source for replay.

Three database files separate operational state, materialized read views and the same-host read replica. Saved trip bundles deliberately combine route points, important events and quality statistics so a selected trip needs one detail fetch. Trip lists are paginated. Redis entries expire quickly and fall back to the replica when Redis is unavailable.

Query	Before ms	After ms	Change
Vehicle detail	Not measured	Not measured	Read saved summary and health statistics
Selected trip	Not measured	Not measured	Fetch one prebuilt route and event bundle

Illustrative sizing assumption: 10,000 vehicles sending one 1 KB event every 30 seconds produce about 333 events/second and 28.8 GB/day of raw payloads before indexes, replicas and backups. At 100,000 vehicles, those figures become about 3,333 events/second and 288 GB/day. These are capacity estimates, not tested capabilities. Retention and archival policies must be set for a pilot.

5.4 Deployment View

Docker Compose runs backend, simulator, a Caddy container serving both built interfaces, and Redis. Only web ports need to be public; backend and simulator ports stay on the internal network. Health checks establish startup readiness. Persistent volumes retain operational and simulator data.

On Windows, Start-ZSpeed.cmd or start.ps1 prepares configuration and starts the stack. start.sh provides the Linux entry point. The AWS helper packages the source and provisions a single EC2 host through CloudFormation. A domain and DNS configuration enable HTTPS through Caddy. Docker, cloud smoke tests and backup restoration remain release checks.

6. Low-Level Design

6.1 Layering & Separation of Concerns

The backend uses routes for request boundaries, services for ingestion, repair and projections, database modules for persistence, and connector adapters for OEM-specific behavior. Some services still use better-sqlite3 directly; a fully database-independent repository layer is future migration work.

Layer	Responsibility	Boundary
Presentation	Fleet and simulator screens; HTTP and socket handlers	Preserve selected views and check user permissions
Application services	Import, connections, ingestion, repair and recovery	Validate changes before publishing a mapping version
Processing	Trip building, route checks, projection jobs	Run outside client page requests
Persistence and adapters	SQLite, Redis, encrypted secrets and OEM connectors	Keep raw history and scope reads to the fleet

Repository layout: frontend/src contains fleet views; simulator-ui/src contains simulator controls; backend/src contains routes, services, database code, simulator code and tests; docker contains deployment configuration. Top-level launch scripts start the packaged environment.

6.2 Design Principles Applied

Idempotency combines event identity with a payload hash. An exact repeat reuses the accepted result; different content with the same identity is held for review. Role and fleet checks enforce least privilege. Environment configuration supplies deployment settings. Dedicated mapping and route functions keep transformation behavior testable.

6.3 Design Patterns Used

Pattern	Purpose	Code
Adapter	Translate OEM-specific formats into canonical readings	connector contract and format mapping
Separate write and read models	Keep vehicle browsing away from raw-event reconstruction	read-model.service.ts
Durable queue with retries	Resume processing and recovery after interruptions	worker.service.ts
Cache bypass and bounded waits	Serve replica data if Redis fails or stalls	read-cache.service.ts

6.4 Interfaces, Contracts & Runtime Flows

Client APIs use JSON and fleet-scoped authorization. Live notifications use Socket.IO. The canonical event retains source identity, event time, vehicle association, supported measurements and quality

information. Mapping versions define source fields, conversions and status translations; no Kafka topics or Avro registry are deployed today.

Normal flow: simulator sends event → backend retains raw payload → identity and format checks → normalized measurements → queued projection → read replica → UI refreshes the affected data.

Failure flow: format fails → category-specific repair → sampled preview → immutable version enabled → durable replay → valid records restored, remaining invalid records held.

6.5 Algorithms & Data Structures

Trip building sorts and deduplicates points, filters implausible GPS jumps, segments by ignition or time gaps and calculates distance using the haversine formula. Sorting costs $O(n \log n)$; linear checks cost $O(n)$. Route simplification uses Ramer Douglas Peucker, which can be $O(n^2)$ in the worst case. Stored bundles avoid repeating this work on every visit. Large-input timing and memory profiles remain to be measured.

7. Non-Functional Requirements & Performance Benchmarks

Requirement	Pilot target	Current result	Verification
API response	p95 below 200 ms under agreed pilot load	Not benchmarked	Load and soak test required
Fresh vehicle view	Show completed snapshots and freshness	Implemented; eventual consistency	Read scaling and browser checks
Recovery	Resume jobs after worker interruption	Covered for durable queue and replay	Focused integration tests
Cache outage	Read from replica when Redis is unavailable	Bounded fallback implemented	Read cache tests; Docker case pending
Availability	99.9 percent service objective	Single host remains a failure point	Multi-host design and failover test required

The template's 100,000 events/second target has not been demonstrated. A repeatable benchmark should record hardware, event mix, fleet size, read/write concurrency, p95 and p99 latency, queue age, replica lag and database growth. Comparison should use the same workload before and after saved read models.

8. Security & Compliance

Authentication and role checks protect fleet APIs and socket rooms. Connection secrets are encrypted in a vault. Production launch scripts create credentials and encryption keys. A domain deployment can use HTTPS; local development and simulated OEM authorization do not establish an OIDC or mTLS deployment.

Threat	Control	Remaining work	Scope
Cross-fleet access	Fleet-scoped queries and socket rooms	Independent security review	Tenant isolation
Unsafe repair	Role checks, previews, immutable mapping versions	Audit review and approval policies for pilots	Mapping changes
Duplicate or changed event identity	Payload hashing and quarantine	Provider correction for true identity conflicts	Ingestion
Credential disclosure	Encrypted vault and redacted source values	Managed key rotation and secrets service	OEM access
Resource exhaustion	Background batches and bounded cache waits	Production load limits and abuse tests	Availability

Location history is sensitive. Before a pilot, define retention, deletion, access review, backup retention and data processing responsibilities. The demonstrator uses synthetic telemetry; GDPR or DPDP compliance certification is not claimed. Full database encryption, regional disaster recovery and penetration testing are future requirements.

9. Test Strategy

Test area	Tools	Checks	Result	CI
Repair backend	Node test runner and TypeScript	27	Passed in focused validation	Not verified
Repair plans and guidance	Node test runner	19	Passed in focused validation	Not verified
Read scaling	Node test runner	22	Passed in earlier validation	Not verified
Grouped activity and refresh	Node test runner	8	Passed in earlier validation	Not verified
End-to-end repair	Browser and isolated synthetic fleet	1 walkthrough	Preview, replay and resolution completed	Manual
Production build	TypeScript and Vite	Backend and frontend	Passed after repair changes	Not verified
Cloud, load and security	Planned	No benchmark count	Docker/Redis container check skipped without Docker	Pending

The latest repair validation passed 46 focused checks. These counts describe separate validation runs, not a freshly rerun complete suite or a line-coverage percentage. Cases cover renamed nested fields, unit conversions, added and removed readings, status changes, GPS pairs, timestamp requirements, identity conflicts, stale drafts, sensitive fields and tenant permissions.

Example: a new OEM payload moves speed to metrics.speed and sends 80 km/h. The manager selects that field and unit, sees 80 → 80 km/h, previews all selected sample records and restores valid readings. Invalid samples stay held. The browser walkthrough also verified a newly added heading, translated engine status and a removed odometer field.

10. Observability

The Background Jobs tab groups continuous processing for the same vehicle under a stable job ID. It shows the purpose, first start, latest activity, status, record counts and recovery progress. Data Pipeline updates preserve the current view and repair form instead of reloading the entire page every two seconds.

To investigate a slow vehicle view, check the snapshot timestamp, find that vehicle's job, inspect its status and record totals, then review worker logs and replica progress. A growing queue suggests processing pressure; fresh projections with slow responses suggest read or cache latency. Distributed traces, latency percentiles and alert thresholds are proposed operational additions.

11. AI / ML Component (if used)

There is no trained model or LLM in the runtime repair path. Suggestions use deterministic field matching and explicit conversion rules. Ambiguous source choices stay with the manager. Future assisted suggestions would require measured accuracy, clear provenance and approval before any mapping is enabled.

12. Architecture Decisions, Risks & Future Enhancements

The current decisions favor a deployable single-host demonstrator with recoverable processing and useful read isolation. The roadmap below describes proposed work; these capabilities are not part of the current deployment.

Decision	Current choice	Consequence
Operational database	SQLite WAL	Simple setup; one host and writer limit
Messaging	Durable SQLite queue	Restart recovery without a broker; multi-host scaling requires replacement
Read consistency	Asynchronous projections and replica	Pages stay responsive; users may see a recently completed snapshot
Repair safety	Previewed immutable mapping versions	Changes remain reviewable and original records are retained
Cache role	Optional short-lived Redis entries	Redis failure can bypass to replica; freshness is bounded by both layers

Future product experience

A manager would open a single attention list showing which OEM changed, how many vehicles are affected and which readings need review. Each item would offer a proposed field or unit change with old and new samples, a confidence indicator and a preview of affected trips. The manager would approve a version, schedule recovery and watch one grouped progress row. An audit history would show who changed what and allow a new version to restore the previous rules.

Stage	Enhancement	Ready when
Pilot readiness	Connect a real OEM; add secure token rotation, audit history, retention and tested backups	An approved real feed passes contract tests and an operator can restore a backup
Scale the data layer	Move operational storage to PostgreSQL with an independent read replica; add an outbox and Kafka or a managed queue	Load tests meet defined read/write targets and crash replay preserves idempotency
Run across hosts	Deploy API and workers on multiple hosts with load balancing, metrics, alerts and controlled autoscaling	Host failure tests meet recovery targets without losing acknowledged events

Stage	Enhancement	Ready when
Improve repair assistance	Offer schema differences and suggested mappings while retaining explicit preview and manager approval	User testing and regression checks show safe, understandable suggestions

Proposed cloud flow: OEM webhooks or polling → authenticated ingestion → transactional outbox → Kafka partitioned by vehicle → normalization and projection workers → PostgreSQL primary and independent read replica → Redis → fleet API. Object storage would hold archived raw payloads under a defined retention policy. Stable identifiers, idempotent consumers and dead-letter handling would keep retries safe.

Kafka versus a simpler managed queue should be decided from ordering, replay, throughput and operating cost needs. PostgreSQL migration must validate counts and hashes, compare old and new read views and support rollback. Availability claims should follow failure testing rather than the choice of a cloud service.

13. Demo Video (5 Minutes Maximum)

Demo video URL: _____

Alternate video URL: _____

Keep the final recording within five minutes. The outline below is a recording plan, not a completed video or a verified timestamp index.

Time	Segment	What to show
0:00–0:30	Problem	A fleet manager sees a format change and held readings
0:30–1:00	Solution	Fleet view, vehicle health and saved trip route
1:00–2:00	Connected data	Import or select vehicles, connect a simulator OEM and show a live event
2:00–3:30	Manager repair	Match a renamed field, choose its unit, preview and restore valid readings
3:30–4:15	Recovery and architecture	Show the grouped recovery job, restored trip data and current queue/read model
4:15–5:00	Evidence and next steps	Show focused test outcomes and the proposed real OEM and distributed deployment

14. Repository Checklist

GitHub repository URL: _____

Release or submission tag URL: _____

- Included: service folders, source tests, Docker Compose, local launch scripts and deployment documentation.
- Before submission: reconcile README architecture with the latest read model, record the demo, add the links and confirm the submission tag.
- Before publishing: run the full suite and container smoke test, configure CI, scan dependencies and images, and verify that generated credentials and database files are excluded.

15. Conclusion

ZSpeed demonstrates a complete path from connected vehicle ingestion to trip visibility and manager-led recovery after supported OEM format changes. Preserved raw history, versioned repairs and saved read views make the results inspectable and recoverable. The next validation step is a real OEM pilot with measured load, security review and tested backup recovery.

16. Declarations

Open-source components include React, Vite, TypeScript, Node.js, Express, better-sqlite3, SQLite, Socket.IO, Leaflet, Redis and Caddy. Exact versions and license obligations should be captured from the lockfiles and deployment images in a submission software bill of materials; no license audit is claimed here.

AI tools: OpenAI Codex assisted with code changes, test development, troubleshooting and this solution document. The deployed application does not call an LLM to normalize or repair telemetry.

Demo data: OEM names and telemetry are generated by the simulator. The demonstrated scenarios do not represent live commercial OEM integrations. Confirm that the final recording, repository and any added dataset contain only permitted synthetic or public data.

Team declaration and approval: _____

17. Appendix (if any)

Implementation references are relative to the repository and describe the current source:

- DEPLOYMENT.md and docker-compose.yml: packaged services, launch configuration and cloud prerequisites.
- READ_SCALING_PLAN.md: saved read views, replica, cache and background processing work.
- frontend/src/components/MappingRepair.tsx and RepairWorkbench.tsx: category-specific manager repair screens.
- backend/src/services/mapping-repair.service.ts and mapping-repair.engine.ts: mapping versions, preview and conversion rules.
- backend/src/services/read-model.service.ts, read-cache.service.ts and worker.service.ts: saved summaries and durable work.
- backend/src/tests/: repair, data quality, tenant isolation, read scaling, queue and trip tests.

Supporting test report URL: _____

Additional documentation URL: _____